Assignment 6

1. Document the best accuracy reached on test-data for both model types (Include numbers in the PDF and deliver the modified code).

   The best accuracy reached on the MPL was 0.9679 and for CNN the best accuracy was 0.9761. See the attached screenshot of the output

Output from MPL

```
(venv) simenaure@simen-macbook Assignement 6 % python assignment_6.py
/Users/simenaure/Desktop/tdt4171/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, rently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 101,770 (397.54 KB)
 Trainable params: 101,770 (397.54 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 851us/step - accuracy: 0.7810 - loss: 0.8384 - val_accuracy: 0.9372 - val_loss: 0.2257
Epoch 2/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 721us/step - accuracy: 0.9288 - loss: 0.2552 - val_accuracy: 0.9578 - val_loss: 0.1672
Epoch 3/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 713us/step - accuracy: 0.9453 - loss: 0.1950 - val_accuracy: 0.9637 - val_loss: 0.1351
Epoch 4/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 711us/step - accuracy: 0.9558 - loss: 0.1540 - val_accuracy: 0.9682 - val_loss: 0.1174
Epoch 5/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 714us/step - accuracy: 0.9643 - loss: 0.1263 - val_accuracy: 0.9698 - val_loss: 0.1103
Epoch 6/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 716us/step - accuracy: 0.9701 - loss: 0.1075 - val_accuracy: 0.9737 - val_loss: 0.0990
Epoch 7/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 765us/step - accuracy: 0.9751 - loss: 0.0927 - val_accuracy: 0.9745 - val_loss: 0.0915
Epoch 8/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 0s 716us/step - accuracy: 0.9781 - loss: 0.0799 - val_accuracy: 0.9757 - val_loss: 0.0853
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 299us/step - accuracy: 0.9679 - loss: 0.1056
(venv) simenaure@simen-macbook Assignement 6 %
```

Output for CNN

```
(venv) simenaure@simen-macbook Assignement 6 % python assignment_6.py
/Users/simenaure/Desktop/tdt4171/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, cur rently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 64)	1,088
max_pooling2d (MaxPooling2D)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 128)	73,856
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 76,234 (297.79 KB)
 Trainable params: 76,234 (297.79 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.6173 - loss: 1.2690 - val_accuracy: 0.9480 - val_loss: 0.1879
Epoch 2/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9313 - loss: 0.2291 - val_accuracy: 0.9683 - val_loss: 0.1169
Epoch 3/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9535 - loss: 0.1547 - val_accuracy: 0.9717 - val_loss: 0.0922
Epoch 4/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9634 - loss: 0.1202 - val_accuracy: 0.9758 - val_loss: 0.0799
Epoch 5/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9691 - loss: 0.1038 - val_accuracy: 0.9763 - val_loss: 0.0755
Epoch 6/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9722 - loss: 0.0893 - val_accuracy: 0.9787 - val_loss: 0.0661
Epoch 7/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9736 - loss: 0.0847 - val_accuracy: 0.9823 - val_loss: 0.0584
Epoch 8/8
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9770 - loss: 0.0745 - val_accuracy: 0.9820 - val_loss: 0.0614
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 653us/step - accuracy: 0.9761 - loss: 0.0770
```

2. After trying a few different model architectures and training settings, for both the MLP and the CNN, write a few sentences comparing the performance achieved for both models. Relate this to the design of the convolutional layer and the total number of parameters for each model.

After experimenting with different architectures and training settings, both the MLP and the CNN achieved the following accuracy on the test data; the MLP reached a test accuracy of 0.968, while the CNN achieved a slightly higher accuracy of about 0.976. Although both models performed well, the CNN reached the best results.

One reason for this difference is the design of the convolutional layer. Convolutional layers use small kernels that slide across the image and learn local patterns such as edges and strokes. Because the same weights are reused across the entire image, CNNs can learn useful features while using fewer parameters.

In this script the CNN used 76,234 parameters, while the MLP used 101,770 parameters, yet the CNN still achieved higher accuracy.

The MLP architecture was simpler, consisting of a flattened input followed by a dense hidden layer. This structure is easy to train and already gives good performance, but it does not explicitly preserve the spatial structure of the image. In contrast, the CNN exploits the spatial relationships between pixels, which makes it more suitable for image classification tasks and explains why it achieved slightly better accuracy with fewer parameters.

3. The model output is a softmax layer with 10 units, and the models are trained with the categorical cross entropy loss. Give a short explanation of how categorical cross entropy on softmax output learns to classify the MNIST-data.

For MNIST, the model outputs 10 numbers (one for each digit 0–9). The softmax layer converts these numbers into probabilities that sum to 1, that represent the probability for each class. Categorical cross-entropy compares this predicted probability distribution with the true label.

During training:

1. The model predicts probabilities with softmax.
2. Cross-entropy loss measures how far the predicted distribution is from the correct label.
   - If the correct class has low probability → high loss.
   - If the correct class has high probability → low loss.

Backpropagation adjusts the network weights to increase the probability of the correct digit and decrease wrong guesses.
Over many examples, the network learns features of the digits so that the correct class gets the highest softmax probability, enabling classification.